# Library Blockchain Lending Tracker — Formative 2

GitHub: https://github.com/relisha1/Individual-Assignment-1

## 1. Introduction

Formative 1 built a blockchain ledger for library lending — every borrow and return became a block, chained together with SHA-256 and signed with ECDSA, so a past record couldn't be edited without it being obvious. This formative adds two things on top of that: a token reward for returning books, and a mining step that has to happen before any of it counts.

The idea is to mirror how a real blockchain works a bit more closely. Right now, when someone returns a book, nothing happens instantly — the record just waits in a pending pool. A miner has to come along, do some proof-of-work, and confirm it before it's added to the permanent chain and the member's balance actually changes. I built two different ways of tracking who owns how many tokens (UTXO and account-based), and three different ways of simulating mining (solo, pool, and cloud), since the assignment wanted a sense of how each one works, not just one mining style.

## 2. How Returning a Book Now Triggers a Token Transaction

Borrowing still works the same as before — the registries get checked, the book gets marked as out. But now, instead of becoming a confirmed block right away, the borrow event goes into a pending pool. It's signed at that point (so no one can later claim it said something different), but it doesn't have a position in the chain yet and it isn't linked to anything.

Returning works the same way. If the book comes back before its due time, the member earns 10 tokens; if it's late, they earn 5. If a book is just never brought back, nothing happens — no block, no transaction, nothing to confirm. The reward amount and a transaction ID (a SHA-256 hash of who, what, and when) get attached to the return block right away, but — and this is the part that's different from Formative 1 — the member doesn't actually receive the tokens yet. That only happens once a miner confirms the block.

One thing worth being upfront about: a real 14-day loan period would make it impossible to demonstrate a late return in a short video, so I set the loan period to 20 seconds instead. It's a clearly marked constant in the code (LOAN_PERIOD_SECONDS) and the README says exactly how to change it back to something realistic.

## 3. The Pending Pool and How Mining Confirms Blocks

The pending pool is just a second list of blocks, separate from the chain. A block sits there — fully signed, but with no index and no previous_hash yet — until it gets mined. Mining is where a block actually gets its place in the chain: its index is set to wherever the chain currently ends, its previous_hash is copied from the last confirmed block, and then the program searches for a nonce that makes the block's hash start with enough zeros (the difficulty).

Once that nonce is found, the block is appended to the real chain. If it was a return with a reward attached, that's the moment the member's balance actually updates — not before. I think this is the part of the assignment that matters most: it shows that confirmation isn't just a formality, it's the thing that makes a transaction real.

## 4. The UTXO Model

UTXO stands for "unspent transaction output," and it's the model Bitcoin uses. Instead of a single number representing someone's balance, tokens exist as individual chunks — outputs from past transactions that haven't been spent yet. A member's balance is just the sum of all the unspent chunks that belong to them.

When a book is returned and confirmed, the reward creates a brand new chunk (a "coinbase" style output, since it's new tokens being minted, not moved from somewhere else) owned by that member. No fee applies here, since there's nothing being spent.

Spending is different. If a member wants to send tokens to someone else, the program gathers up their unspent chunks until it has enough to cover the amount plus a 1-token fee. If there's more than needed, the leftover comes back to the sender as a new "change" chunk — the same idea as getting change back from a cash payment. If they don't have enough chunks to cover

it, the transfer is rejected outright. Every chunk that gets used is marked spent so it can never be used again, which is what stops double-spending.

# 5. The Account-Balance Model

This one is simpler to think about, and it's closer to how a bank account or Ethereum works. Each member just has one number: their balance. A reward adds directly to it. A transfer checks that the sender has enough, then subtracts from one balance and adds to the other.

The one extra piece is the nonce. Every account has a counter that starts at 0, and every outgoing transfer has to use the exact next expected number. If someone tries to reuse an old nonce or skip ahead, the transfer is rejected. This is the same idea real blockchains use to stop someone from replaying an old, already-processed transaction. I also keep a running history for each account — a linked list of every transfer that account was part of, including the reward credits — so there's a full paper trail for any member.

# 6. Comparing the Two Models

| | **UTXO** | **Account balance** |
|---|---|---|
| How balance is found | sum of unspent outputs | a single stored number |
| Stops double-spending | marking outputs as spent | the nonce counter |
| Change | an explicit change output | not needed — balance just updates |
| History | scattered across many outputs | one linked list per account |
| Easier to reason about? | more moving parts, closer to real Bitcoin | simpler, closer to a bank account |

Neither one is "better" exactly — they trade off differently. UTXO makes it very easy to prove exactly where every token came from, since each one traces back to a specific transaction, but it means more bookkeeping (you can end up with lots of small chunks). Account balances are easier to follow at a glance and need less data, but you lose that direct traceability unless you keep a separate log, which is why I added the transaction history list for that model.

# 7. How the Mining Simulations Work

## 7.1 Solo mining — real proof-of-work

This one isn't simulated at all — it's the real thing, just at a small scale. For each pending block, the program tries nonce = 0, 1, 2, and so on, hashing the block each time, until the resulting hash starts with however many zero hex digits the difficulty asks for (default 2). It counts every attempt and shows that number once it succeeds. Raising the difficulty by one roughly multiplies the expected attempts by sixteen, which is easy to see just by bumping it up during a demo.

## 7.2 Pool mining — shared real work, simulated split

Pool mining still does the same real proof-of-work search for every pending block, so the total attempt count is genuine. What's simulated is how that work gets divided: I generate a handful of fake miners, each given a random "hash power" weight, and split the real attempt count between them proportionally to those weights. A 2% pool fee comes off the top of the block reward first, and what's left is shared out the same way — more weight means a bigger share of both the attempts and the reward. The result is printed as a table.

## 7.3 Cloud mining — rental economics

Cloud mining is framed differently on purpose: in real life, a cloud miner doesn't do any hashing themselves, they just pay for someone else's. So this mode runs for however many rounds the user rents (1 to 5), and each round charges a fixed rental fee and credits a reward (which varies, since real mining payouts aren't constant). If there's still something in the pending pool, that round actually mines and confirms a real block behind the scenes. The program tracks gross earnings, total fees, and net profit across all rounds, and prints a warning the moment cumulative fees pass cumulative rewards — which happens more often than you'd think, which is sort of the point of the warning.

# 8. Design Choices and Assumptions

- Loan period is 20 seconds for demo purposes, documented clearly in both the code and the README, with the real 14-day value left as a comment.
- The miner's own reward (solo/pool/cloud) is displayed but not credited to any member account, since a miner isn't necessarily a library member — it's shown to demonstrate the mechanism, not to model miner identity.
- UTXO transfers made during a session aren't written to blockchain.dat — only lending blocks are. On restart, reward credits reappear because they're replayed from the chain, but manual transfers from the previous session don't carry over. I'm noting this here rather than hiding it, since it's a real limitation of the current persistence approach.
- The ledger model (UTXO vs account) is chosen once at startup and applies for that whole session, rather than letting someone switch mid-run, since mixing the two models on the same data would be confusing to reason about.

## 9. Screenshots

UTXO model: borrowing, mining a return on time, viewing the confirmed chain, the UTXO set, and a transfer with change returned:

```
Loading registries...
Loaded 6 book(s) and 5 member(s).
Choose the token ledger model for this session:
  1. UTXO (inputs/outputs, like Bitcoin)
  2. Account balances with nonces (like Ethereum)
Enter 1 or 2: [key] Generated new library signing key -> library_key.der
Starting a fresh chain (first mined block will be the genesis block).

Choose an option: Book ID: Member ID: OK: 'Things Fall Apart' borrowed by John Doe. Added to the pending pool (not
yet mined). Due in 20 seconds.

Choose an option:
Pending pool (1 block(s) waiting to be mined):
  [0] BORROWED   Things Fall Apart      member ALU001     reward 0

Choose an option: Book ID: Member ID: OK: 'Things Fall Apart' returned by John Doe (ON TIME). Reward of 10 tokens
queued as tx 6decdea49a... Added to the pending pool — balance updates only after mining.

Choose an option:
--- Solo mining (difficulty 2) ---
Block #0 (BORROWED, Things Fall Apart) mined after 538 attempts. Hash: 0038a178bff80f01...
Block #1 (RETURNED, Things Fall Apart) mined after 54 attempts. Hash: 00a81115e84adffd...
Mined 2 block(s), 592 total hash attempts, solo miner reward: 100 tokens (reported only — not credited to any member
account).

Choose an option:
IDX  ACTION     REWARD     BOOK                   MEMBER     SIG      TX ID
-------------------------------------------------------------------------------
0    BORROWED   0          Things Fall Apart      ALU001     valid    -
1    RETURNED   10         Things Fall Apart      ALU001     valid    6decdea49a

Choose an option:
OWNER      OUT# AMOUNT     SPENT    TX ID (first 12 chars)
-------------------------------------------------------
ALU001     0    10         no       6decdea49aba

Balances (unspent only):
  ALU001     10
  ALU002     0
  ALU003     0
  ALU004     0
  ALU005     0

Choose an option: Sender member ID: Recipient member ID: Amount: OK: ALU001 sent 3 to ALU002 (fee 1). 6 returned to
ALU001 as change.

Choose an option:
OWNER      OUT# AMOUNT     SPENT    TX ID (first 12 chars)
-------------------------------------------------------
ALU001     0    10         yes      6decdea49aba
ALU002     0    3          no       61c0ea63192a
ALU001     1    6          no       61c0ea63192a

Balances (unspent only):
  ALU001     6
  ALU002     3
  ALU003     0
  ALU004     0
  ALU005     0

Choose an option: Goodbye.
```

Account model: an overdue loan being flagged, a late return earning the reduced reward, a nonce-based transfer, and the resulting transaction history:

```
Loading registries...
Loaded 6 book(s) and 5 member(s).
Choose the token ledger model for this session:
  1. UTXO (inputs/outputs, like Bitcoin)
  2. Account balances with nonces (like Ethereum)
Enter 1 or 2: [key] Generated new library signing key -> library_key.der
Starting a fresh chain (first mined block will be the genesis block).

Choose an option: Book ID: Member ID: OK: 'Americanah' borrowed by Jane Smith. Added to the pending pool (not yet
mined). Due in 20 seconds.

Choose an option:
Active loans (borrowed, not yet returned):
  BK002    Americanah               held by ALU002     [within loan period]

Choose an option:
Active loans (borrowed, not yet returned):
  BK002    Americanah               held by ALU002     [OVERDUE — no reward until returned]

Choose an option: Book ID: Member ID: OK: 'Americanah' returned by Jane Smith (LATE). Reward of 5 tokens queued as
tx aeb27508a0... Added to the pending pool — balance updates only after mining.

Choose an option:
--- Solo mining (difficulty 2) ---
Block #0 (BORROWED, Americanah) mined after 134 attempts. Hash: 00f0d682309aa6a8...
Block #1 (RETURNED, Americanah) mined after 406 attempts. Hash: 0067b3c107c7b288...
Mined 2 block(s), 540 total hash attempts, solo miner reward: 100 tokens (reported only — not credited to any member
account).

Choose an option:
MEMBER     BALANCE    NEXT NONCE
---------------------------------
ALU001     0          0
ALU002     5          0
ALU003     0          0
ALU004     0          0
ALU005     0          0

Choose an option: Sender member ID: Recipient member ID: Amount: Nonce for this transfer: OK: ALU002 sent 2 to
ALU003 (fee 1, nonce 0). New balance: 2.

Choose an option: Member ID:
Transaction history for ALU002:
  SYSTEM -> ALU002 | amount 5 | fee 0 | nonce 0
  ALU002 -> ALU003 | amount 2 | fee 1 | nonce 0

Choose an option: Goodbye.
```

Pool mining's reward-sharing table, cloud mining running into an unprofitable round, and tamper detection catching an edited block then confirming the chain is valid again once restored:

```
Loading registries...
Loaded 6 book(s) and 5 member(s).
Choose the token ledger model for this session:
  1. UTXO (inputs/outputs, like Bitcoin)
  2. Account balances with nonces (like Ethereum)
Enter 1 or 2: [key] Generated new library signing key -> library_key.der
Starting a fresh chain (first mined block will be the genesis block).

Choose an option: Book ID: Member ID: OK: 'The River Between' borrowed by John Doe. Added to the pending pool (not
yet mined). Due in 20 seconds.

Choose an option: Book ID: Member ID: OK: 'Half of a Yellow Sun' borrowed by Kwame Mensah. Added to the pending pool
(not yet mined). Due in 20 seconds.

Choose an option: How many simulated pool miners (2-6)?
--- Pool mining 2 block(s) (difficulty 2) ---
Total real hash attempts across all mined blocks: 258
Block reward pool: 100 tokens, pool fee (2%): 2.00, net to distribute: 98.00

MINER      ATTEMPTS     SHARE %    REWARD
-----------------------------------------------
miner_1    4            1.83       1.80
miner_2    118          45.87      44.95
miner_3    134          52.29      51.25

Choose an option: Book ID: Member ID: OK: 'So Long a Letter' borrowed by Fatima Ndoye. Added to the pending pool
(not yet mined). Due in 20 seconds.

Choose an option: Rental duration in rounds (1-5)?
--- Cloud mining: 2 round(s), difficulty 2, rental fee 6/round ---
Round 1: mined block #2 (BORROWED) in 204 attempts. Fee -6, reward +10.
Round 2: no pending blocks left to mine. Fee -6, reward +0.
  WARNING: cumulative fees (12) exceed cumulative rewards (10) — this rental is unprofitable so far.

Summary: gross earnings 10, total fees 12, net profit -2.

Choose an option: Enter the index of a confirmed block to tamper with (1-2):
--- BEFORE tampering ---
Block #0   hash:OK  link:OK  signature:VALID
Block #1   hash:OK  link:OK  signature:VALID
Block #2   hash:OK  link:OK  signature:VALID

[Changing block #1's token_reward without re-mining or re-signing it]

--- AFTER tampering ---
Block #0   hash:OK  link:OK  signature:VALID
Block #1   hash:MISMATCH  link:OK  signature:INVALID
Block #2   hash:OK  link:OK  signature:VALID

Chain validation result: TAMPERING DETECTED

Choose an option: Block #0   hash:OK  link:OK  signature:VALID
Block #1   hash:OK  link:OK  signature:VALID
Block #2   hash:OK  link:OK  signature:VALID

Overall chain status: VALID

Choose an option: Goodbye.
```

# 10. Test Results for Edge Cases

| Case | What I tested | Result |
|---|---|---|
| Book never returned | Left a book borrowed and just checked option 5 without returning it | Shown as an active/overdue loan; no transaction, no block, exactly as expected |
| Insufficient balance | Tried to send more tokens than a member had (both models) | Rejected with the exact shortfall shown |
| Reused/incorrect nonce | Sent a transfer with the wrong nonce under the account model | Rejected: "ERROR: wrong nonce. Expected X, got Y." |
| Unprofitable cloud rental | Ran cloud mining with rounds where the pending pool ran dry partway through | Fee still charged with no reward that round, and the program printed the unprofitability warning once fees passed rewards |

| | | |
|---|---|---|
| Tampering | Edited a confirmed block's token_reward directly in memory, without re-mining or re-signing it | Hash mismatch and invalid signature caught immediately by chain validation |

# 11. System Design

This lays out the full flow — how a borrow or return becomes a pending block, how mining turns it into a confirmed one, and how that confirmation feeds into whichever ledger model is active.

